

20. Principy vrstvené architektury počítačových sítí, referenční model OSI. Charakteristika lokálních počítačových sítí.

Principy vrstvené architektury

Síťová komunikace je komplexní. **Vrstvená architektura** ji rozděluje na menší, spravovatelné vrstvy, z nichž každá má specifické funkce.

Hlavní principy a výhody: * **Dekompozice problému:** Rozdělení složitého úkolu na jednodušší vrstvy. *

Abstrakce a rozhraní: Každá vrstva poskytuje služby vyšší vrstvě a využívá služby nižší. Interní detaily jsou skryté. Standardizovaná rozhraní umožňují nezávislý vývoj. * **Peer-to-peer komunikace:** Stejně vrstvy protilehlých systémů komunikují virtuálně pomocí **protokolů**. * **Zjednodušení údržby:** Snadnější identifikace a řešení problémů.

Zapouzdření (Encapsulation) a Odpouzdrnění (Decapsulation): * **Zapouzdření:** Při odesílání dat (shora dolů) každá vrstva přidá svou řídicí informaci – **hlavičku (Header)**, případně **patičku (Trailer)**. Vzniká **Protocol Data Unit (PDU)**. * **Odpouzdrnění:** Na přijímací straně (zdola nahoru) každá vrstva svou hlavičku/patičku odřízne a zpracuje.

Příklad zapouzdření: 1. Aplikační data: DATA 2. Transportní vrstva: [TCP_H] DATA (Segment) 3. Síťová vrstva: [IP_H] [TCP_H] DATA (Paket) 4. Linková vrstva: [ETH_H] [IP_H] [TCP_H] DATA [ETH_T] (Rámec) 5. Fyzická vrstva: Převeď na bity a odešle.

Referenční model OSI (Open Systems Interconnection)

Teoretický, sedmivrstvý model ISO pro standardizaci síťových protokolů. V reálném Internetu se používá model TCP/IP, ale OSI je klíčový pro výuku.

Vrstvy modelu OSI (od nejvyšší po nejnižší):

7. Aplikační vrstva (Application Layer)

Účel: Poskytuje síťové služby přímo uživatelským aplikacím.

Protokoly: HTTP, FTP, SMTP, DNS.

PDU: Data.

6. Prezentační vrstva (Presentation Layer)

Účel: Zajišťuje, že data jsou pro aplikaci čitelná (formát, kódování, komprese, šifrování).

Protokoly: JPEG, MPEG, TLS/SSL.

PDU: Data.

5. Relační vrstva (Session Layer)

Účel: Správa, navazování, udržování a ukončování dialogů (relací) mezi aplikacemi.

Protokoly: NetBIOS, RPC.

PDU: Data.

4. Transportní vrstva (Transport Layer)

Účel: Zajišťuje transparentní přenos dat mezi koncovými aplikacemi (End-to-End).

Funkce: Adresování aplikací pomocí **portů**. Poskytuje spolehlivý (TCP) nebo nespolehlivý (UDP) přenos.

Protokoly: TCP (Transmission Control Protocol), UDP (User Datagram Protocol).

PDU: Segment (TCP) / Datagram (UDP).

3. Síťová vrstva (Network Layer)

Účel: Zajišťuje doručení dat mezi dvěma uzly v různých sítích (inter-network).

Funkce: Logické adresování pomocí **IP adres, směrování (Routing)**.

Protokoly: IP (IPv4, IPv6), ICMP.

Zařízení: Router (směrovač).

PDU: Paket.

2. Linková vrstva (Data Link Layer)

Účel: Zajišťuje spolehlivý přenos dat v rámci jedné lokální sítě (LAN) mezi přímo sousedícími uzly (intra-network).

Funkce: Fyzické adresování pomocí **MAC adres, řízení přístupu k médiu, detekce chyb.**

Protokoly: Ethernet (IEEE 802.3), Wi-Fi (IEEE 802.11), PPP.

Zařízení: Switch (přepínač), síťová karta (NIC).

PDU: Rámec (Frame).

1. Fyzická vrstva (Physical Layer)

Účel: Specifikuje fyzické, elektrické, optické a funkční vlastnosti přenosového média.

Funkce: Převádí rámce na bity a moduluje je do fyzických signálů.

Média: Kroucená dvojlinka, optický kabel, rádiový prostor.

Zařízení: Hub (rozbočovač), repeater (opakovač), síťová karta (NIC).

PDU: Bit.

Charakteristika lokálních počítačových sítí (LAN)

LAN (Local Area Network): Síť pro omezenou geografickou oblast (budova, kampus).

Klíčové vlastnosti: * Vysoká rychlost přenosu dat. * Nízká chybovost. * Sdílené přenosové médium.

Klíčové prvky a pojmy: * **MAC adresa (Media Access Control):** Fyzická adresa síťové karty (48 bitů), unikátní, používá se na linkové vrstvě (L2). * **IP adresa (Internet Protocol):** Logická adresa zařízení v síti, používá se na síťové vrstvě (L3) pro směrování. * **Port:** Číslo identifikující konkrétní aplikaci/slужbu na hostiteli, používá se na transportní vrstvě. * **Switch (přepínač):** Zařízení linkové vrstvy (L2), přeposílá datové rámce na základě MAC adresy cíle, učí se MAC adresy připojených zařízení. * **Router (směrovač):** Zařízení síťové vrstvy (L3), směruje datové pakety mezi různými sítěmi na základě IP adres. * **Topologie sítě:** Fyzické nebo logické uspořádání prvků sítě (např. hvězda, sběrnice, kruh). Dnes převládá hvězda se switchem.

21. Technologie Ethernet, její principy a vývoj, algoritmus CSMA/CD. Bezdrátové lokální sítě standardu IEEE 802.11.

Klíčové pojmy

Ethernet (IEEE 802.3): Drátové LAN, vrstvy 1 a 2 OSI.

MAC adresa: Unikátní 48bitová fyzická adresa síťového adaptéru.

Rámec: Základní datová jednotka Ethernetu.

CSMA/CD: Přístupový mechanismus pro sdílený Ethernet (detekce a řešení kolizí).

Wi-Fi (IEEE 802.11): Bezdrátové LAN (WLAN).

CSMA/CA: Přístupový mechanismus pro Wi-Fi (předcházení kolizím).

Half-Duplex: Vysílání nebo příjem, ne současně.

Full-Duplex: Současné vysílání i příjem.

Kolizní doména: Segment sítě, kde může dojít ke kolizím.

A. Technologie Ethernet a její vývoj

Ethernet (IEEE 802.3) je dominantní drátová LAN technologie.

MAC adresa: 48bitový identifikátor síťového rozhraní.

Rámec: Datová jednotka s hlavičkou (MAC adresy), daty a zápatím (FCS).

Vývoj Ethernetu:

Klasický (sdílený) Ethernet (s Hubem):

Sdílené médium, jedna kolizní doména.

Half-Duplex provoz.

Používal **CSMA/CD** pro řízení přístupu a řešení kolizí.

Moderní (přepínaný) Ethernet (se Switchem):

Switch izoluje provoz, každý port je samostatná kolizní doména.

Full-Duplex komunikace, eliminuje kolize.

CSMA/CD se aktivně nepoužívá.

B. Algoritmus CSMA/CD (Carrier Sense Multiple Access with Collision Detection)

Používán v klasickém sdíleném Ethernetu.

Jak funguje:

CS (Carrier Sense): Stanice naslouchá médiu, zda je volné. Pokud ne, čeká.

MA (Multiple Access): Více stanic má stejné právo k přístupu.

CD (Collision Detection): Během vysílání stanice monitoruje médium. Pokud dojde ke kolizi (současné vysílání), detekuje ji.

Řešení kolize:

Jam Signal: Stanice vyšle varovný signál.

Exponential Backoff: Stanice si vygeneruje náhodný časový interval čekání.

Opakování vysílání: Po uplynutí čekací doby se pokusí vysílat znovu.

C. Bezdrátové sítě IEEE 802.11 (Wi-Fi)

Standard pro WLAN. Rádiový prostor je sdílené médium, **Half-Duplex** provoz.

Problém skryté stanice (Hidden Node Problem)

Stanice se navzájem neslyší, ale obě slyší AP. Mohou vysílat současně k AP, což způsobí kolizi u AP, o které se vysílající stanice nedozví.

Algoritmus CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance)

Snaží se kolizím předcházet, protože detekce je obtížná. 1. **Naslouchání kanálu**: Zda je volný. 2. **Čekání a náhodný odstup**: Po zjištění volného kanálu se čeká povinná doba (IFS) a náhodná doba (Contention Window). 3. **Potvrzování (ACK)**: Příjemce musí potvrdit doručení rámce. Pokud odesílatel neobdrží ACK, předpokládá kolizi/ztrátu a zkusí vysílat znovu (s Exponential Backoff).

Volitelný mechanismus RTS/CTS (Request to Send / Clear to Send)

Řeší problém skryté stanice. 1. **RTS**: Vysílající stanice pošle AP dotaz. 2. **CTS**: AP odpoví CTS, který slyší všechny stanice v dosahu AP. 3. **Rezervace média**: CTS explicitně přikáže ostatním stanicím ztichnout, čímž se médium rezervuje pro vysílající stanici.

Přehled základních generací Wi-Fi:

802.11n (Wi-Fi 4): 2.4/5 GHz, MIMO, stovky Mb/s.

802.11ac (Wi-Fi 5): 5 GHz, MU-MIMO, až Gb/s.

802.11ax (Wi-Fi 6): 2.4/5/6 GHz, OFDMA, vylepšené MU-MIMO, efektivita v zahuštěných prostorech.

Nejčastější chyby u zkoušky

Moderní switchovaný Ethernet NEPOUŽÍVÁ kolize. CSMA/CD je pro half-duplex na sdíleném médiu.

MAC adresa (vrstva 2, fyzická) vs. **IP adresa** (vrstva 3, logická).

Wi-Fi médium je VŽDY sdílené, proto CSMA/CA a RTS/CTS.

Typické zkouškové otázky

Popište Ethernetový rámec.

Základní datová jednotka linkové vrstvy. Obsahuje: Preamble, Cílová MAC, Zdrojová MAC, Typ/Délka, Data (46 – 1500 bytů), FCS (kontrola chyb).

Jak funguje CSMA/CD?

Mechanismus pro sdílený Ethernet. Stanice:

Carrier Sense: Naslouchá, zda je médium volné.

Multiple Access: Více stanic má přístup.

Collision Detection: Detekuje kolizi během vysílání.

Řešení kolize: Vyšle **JAM signal**, počká náhodný čas (**Exponential Backoff**) a zkusí znovu.

Proč Wi-Fi používá CSMA/CA?

Efektivní detekce kolizí je v bezdrátovém prostředí obtížná kvůli:

Problému skryté stanice: Stanice se navzájem neslyší, ale kolize mohou nastat u AP.

Half-Duplex provozu: Nelze současně vysílat a detekovat kolize.

CSMA/CA kolizím předchází (naslouchání, náhodné čekání, ACK, volitelně RTS/CTS).

22. Základní principy činnosti protokolů sítě Internet

A. Protokol IP (Internet Protocol)

Síťová vrstva (3. OSI).

Úkol: Nespolehlivé, bezspojové doručování datových paketů mezi uzly.

Bezspojový: Bez navázání spojení, pakety nezávislé.

Nespolehlivý (Best-Effort): Negarantuje doručení, pořadí, integritu. Ztracené pakety se zahodí.

Funkce:

Logické adresování: Unikátní IP adresa.

IPv4: 32bit, např. 192.168.1.1 .

IPv6: 128bit, např. 2001:db8::1 . Větší adresní prostor, zjednodušená hlavička.

Směrování (Routing): Routery předávají pakety dle cílové IP.

Fragmentace: Rozdělení paketu (IPv6 jen odesílatel).

IP hlavička (klíčová pole):

Verze: IPv4/IPv6.

TTL (Time To Live): Počet skoků. Každý router sníží o 1. Při $TTL = 0$ paket zahozen (prevence smyček).

Protokol: Protokol vyšší vrstvy (6=TCP, 17=UDP).

Zdrojová/Cílová IP adresa.

Typické zkouškové otázky: 1. **Úkol IP, vrstva OSI?** Nespolehlivé, bezspojové doručování paketů. Síťová vrstva (3.). 2. **"Bezspojový" a "nespolehlivý" u IP?** Bez navázání spojení, pakety nezávislé; negarantuje doručení, pořadí, integritu. 3. **Rozdíl IPv4 a IPv6?** IPv4 32bit, IPv6 128bit. IPv6 větší adresní prostor, zjednodušená hlavička, fragmentace jen u odesílatele.

B. Transportní protokoly: TCP vs. UDP

Transportní vrstva (4. OSI).

Porty: 16bit čísla pro identifikaci aplikací (HTTP 80, DNS 53).

1. TCP (Transmission Control Protocol)

Charakteristika: Spolehlivý, spojovaný protokol. Zajišťuje doručení, pořadí, integritu.

Mechanismy:

Three-Way Handshake: Navázání spojení.

Klient: SYN

Server: SYN-ACK

Klient: ACK

Potvrzování (ACK): Příjemce potvrzuje segmenty. Bez ACK se data opakují.

Řazení (Sequencing): Sekvenční čísla pro správné pořadí.

Řízení toku (Flow Control): Okno příjemce (Window Size) zabraňuje zahlcení.

Řízení zahlcení (Congestion Control): Zpomalení přenosu při zahlcení sítě.

Použití: HTTP/HTTPS, SMTP, FTP, SSH (kde je spolehlivost klíčová).

2. UDP (User Datagram Protocol)

Charakteristika: Nespolehlivý, bezspojový protokol s minimální režii.

Princip: Odesílá data bez záruk.

Výhody: Extrémní rychlost, nízká latence, nízká režie.

Použití: Streamování (video/audio), online hry, VoIP, DNS dotazy (kde je rychlost klíčová, ztráta dat přijatelná).

Typické zkouškové otázky: 1. **TCP vs UDP (spolehlivost, spojení, režie)?** TCP spolehlivý, spojovaný, vyšší režie. UDP nespolehlivý, bezspojový, minimální režie. 2. **Three-Way Handshake u TCP?** SYN (klient) -> SYN-ACK (server) -> ACK (klient). 3. **Příklady aplikací TCP/UDP a proč?** - **TCP:** Web, e-mail (vyžadují kompletní data). - **UDP:** Streamování, hry, VoIP, DNS (klíčová nízká latence, ztráta dat přijatelná).

C. Domain Name System (DNS)

Role: Distribuovaná, hierarchická databáze pro překlad doménových jmen na IP adresy a naopak.

Hierarchie:

Kořenové servery (.): Vrchol, znají TLD servery.

TLD servery (.cz , .com): Znájí autoritativní servery pro domény 2. řádu.

Autoritativní servery: Drží finální záznamy pro konkrétní doménu.

Local DNS Resolver (Caching Name Server): Klientův server (ISP, Google DNS). Provádí rekurzivní dotazy od klientů a iterativní do hierarchie. Cache.

Postup řešení DNS dotazu:

Klient kontroluje lokální cache.

Klient pošle **rekurzivní dotaz** na Local DNS Resolver.

Local DNS Resolver provádí **iterativní dotazy**:

Dotaz na Kořenový server -> odkaz na TLD server.

Dotaz na TLD server -> odkaz na Autoritativní server.

Dotaz na Autoritativní server -> vrátí IP adresu.

Local DNS Resolver uloží výsledek do cache (dle TTL) a předá klientovi.

Typy DNS záznamů (Resource Records):

A: Doména na **IPv4** (`tu1.cz IN A 147.230.1.2`).

AAAA: Doména na **IPv6** (`tu1.cz IN AAAA 2001:db8::1`).

CNAME: Alias pro jinou doménu (`www.tu1.cz IN CNAME tu1.cz`).

MX: Poštovní servery pro doménu (`tu1.cz IN MX 10 mail.tu1.cz`).

NS: Autoritativní DNS servery pro doménu (`tu1.cz IN NS ns1.tu1.cz`).

PTR: Pro reverzní DNS (IP na doménu).

TXT: Libovolný text (SPF, DKIM).

Reverzní DNS (Reverse DNS lookup):

Úkol: Překlad IP adresy na doménové jméno.

Princip: Používá speciální domény `in-addr.arpa` (IPv4) / `ip6.arpa` (IPv6) a hledá **PTR záznam**.

Využití: Anti-spam (ověření odesílatele e-mailu), logování, diagnostika sítě.

Typické zkouškové otázky: 1. **Role a hierarchie DNS?** Překlad domén/IP. Hierarchie: Kořenové -> TLD -> Autoritativní -> Local Resolver. 2. **Postup řešení DNS dotazu?** Klient cache -> rekurzivní dotaz na Local DNS -> Local DNS iterativně (Kořen -> TLD -> Autoritativní) -> Autoritativní odpoví Local DNS -> Local DNS odpoví klientovi. 3. **Reverzní DNS a využití?** Překlad IP na doménu (PTR záznamy v `in-addr.arpa` / `ip6.arpa`). Použití: Anti-spam, logování, diagnostika. 4. **3 typy DNS záznamů?** A (IPv4), AAAA (IPv6), CNAME (alias), MX (mail), NS (name servery).

23. Základní principy WWW, HTTP, HTML

A. Základní principy WWW (World Wide Web)

Definition: Globální, distribuovaný informační systém pro sdílení hypertextových dokumentů a zdrojů přes internet. Vytvořen Timem Bernersem-Leem (1989, CERN).

Architektura (Klient-Server):

Klient (Webový prohlížeč): Odesílá požadavky, přijímá data, vykresluje.

Server (Webový server): Naslouchá, zpracovává požadavky, posílá odpovědi.

Tři pilíře WWW:

URI / URL: Jednotné adresování a lokalizace zdrojů (např. `https://tul.cz/index.html`).

HTTP / HTTPS: Protokol pro přenos webových zdrojů mezi klientem a serverem.

HTML: Značkovací jazyk pro strukturu a sémantiku webových dokumentů.

B. Protokol HTTP (Hypertext Transfer Protocol)

Definition: Aplikační protokol (7. vrstva OSI) pro přenos hypertextových dokumentů a dat, pracuje nad TCP.

Charakteristiky:

Bezstavový (Stateless): Server si nepamatuje předchozí interakce. Stav se udržuje pomocí **Cookies** a **Sessions**.

Textový (HTTP/1.1): Lidsky čitelná komunikace.

Binární (HTTP/2, HTTP/3): Efektivnější přenos, multiplexing.

Struktura komunikace: Požadavek (Request) a Odpověď (Response). Obě části obsahují úvodní řádek, hlavičky (metadata) a volitelné tělo (data).

HTTP Metody (v požadavku): Určují typ operace.

GET : Získání dat (idempotentní, bezpečný).

POST : Odeslání dat ke zpracování (není idempotentní/bezpečný).

PUT : Nahrazení celého zdroje (idempotentní).

PATCH : Částečná aktualizace zdroje.

DELETE : Smazání zdroje (idempotentní).

HEAD : Získání pouze hlaviček odpovědi.

OPTIONS : Zjištění podporovaných metod.

HTTP Stavové kódy (v odpovědi): Číselně vyjadřují výsledek operace.

1xx : Informativní.

2xx : Úspěch (např. `200 OK` , `201 Created`).

3xx : Přesměrování (např. `301 Moved Permanently` , `304 Not Modified`).

4xx : Chyba klienta (např. `400 Bad Request` , `404 Not Found` , `403 Forbidden`).

5xx : Chyba serveru (např. `500 Internal Server Error` , `503 Service Unavailable`).

REST: HTTP metody jsou základem pro návrh RESTful API.

C. Jazyk HTML a (X)HTML

Definition: Značkovací jazyk pro definici struktury a sémantiky webové stránky pomocí tagů (např. `<h1>` , `<p>`).

`<!DOCTYPE html>` : Deklarace typu dokumentu, zajišťuje "standards mode".

Vývoj:

Klasické HTML (do 4.01): Tolerantní k chybám, nekonzistentní vykreslování.

XHTML: Striktní XML varianta HTML. Vyžadovalo precizní syntaxi (malá písmena, uvozovky pro atributy, uzavřené tagy jako `<br />`). Cíl: robustnost, strojové zpracování. V praxi příliš rigidní.

HTML5: Moderní standard, kombinuje pragmatismus s novými funkcemi.

Méně striktní než XHTML, podporuje validní kód.

Sémantické tagy: `<header>`, `<nav>`, `<article>`, `<section>`, `<footer>`, `<aside>` – zlepšují přístupnost, SEO, čitelnost.

Nativní podpora multimédií (`<audio>`, `<video>`).

Nová API pro JavaScript (`<canvas>`, Geolocation, Web Storage).

Vylepšené formuláře.

Document Object Model (DOM): Stromová struktura HTML dokumentu, kterou prohlížeč vytvoří a kterou lze manipulovat pomocí JavaScriptu.

Možnosti HTML:

Definuje logickou strukturu textu (nadpisy, odstavce, seznamy, tabulky).

Vytváří hypertextové odkazy (`<a href="...">`).

Umožňuje vkládat externí objekty (obrázky, videa) a formuláře.

Omezení HTML (Separace zájmů):

Není programovací jazyk: Nemá logiku, proměnné, podmínky.

Neřeší vzhled: Vizuální prezentace je role **CSS**.

Statičnost: Pro dynamiku na straně klienta je nutný **JavaScript**.

Typické zkouškové otázky (Zkrácené odpovědi)

Základní pilíře WWW a princip klient-server architektury:

Pilíře: URI/URL (adresování), HTTP/HTTPS (přenos), HTML (struktura).

Klient-Server: Klient (prohlížeč) posílá požadavky, server (webový server) je zpracovává a posílá odpovědi.

Protokol HTTP, jeho charakteristiky a struktura komunikace:

HTTP: Aplikační protokol pro přenos hypertextu.

Charakteristiky: Bezstavový (stav přes Cookies/Sessions), textový (HTTP/1.1) / binární (HTTP/2, HTTP/3).

Struktura: Požadavek (metoda: GET, POST, PUT, DELETE; hlavičky) a Odpověď (stavový kód: 2xx úspěch, 4xx chyba klienta, 5xx chyba serveru; hlavičky).

Porovnání HTML, XHTML a HTML5:

HTML (do 4.01): Tolerantní k chybám, nekonzistentní.

XHTML: Striktní XML syntaxe, rigidní.

HTML5: Moderní, pragmatický, sémantické tagy, multimédia, API.

Možnosti HTML: Struktura, odkazy, vkládání objektů, formuláře.

Omezení HTML: Není programovací jazyk, neřeší vzhled (CSS), statické (JS).

Bezstavovost HTTP a udržení stavu:

Bezstavovost: Každý požadavek je nezávislý, server si nepamatuje kontext.

Udržení stavu: Řeší se pomocí **Cookies** (data na klientovi) a **Sessions** (stav na serveru s ID na klientovi).

Sémantické vs. nesémantické HTML tagy a jejich důležitost v HTML5:

Nesémantické: `<div>`, `<span>` – bez významu pro obsah.

Sémantické: `<header>`, `<nav>`, `<article>`, `<footer>` – dodávají obsahu význam a strukturu.

Důležitost: Zlepšují přístupnost (čtečky obrazovky), SEO (vyhledávače), čitelnost kódu a stylování.

24. CSS vlastnosti, hodnoty, dědění, kaskádování.

Blokový model CSS.

Klíčové pojmy

selektor, vlastnost, hodnota, kaskáda, specifita, dědění, blokový model (box model), margin, padding, border, content, box-sizing, display (block, inline, flex)

A. CSS Vlastnosti a Hodnoty

CSS (Cascading Style Sheets) popisuje vzhled HTML dokumentů. **Struktura pravidla:** selektor { vlastnost: hodnota; } * **Selektor:** Určuje elementy (p , .trida , #identifikator). * **Deklarace:**

vlastnost: hodnota; (např. color: red;).

Typy hodnot: * **Délkové jednotky:** * **Absolutní:** px . * **Relativní:** em (k rodiči), rem (k <html>), % , vw / vh (k oknu). * **Barvy:** Klíčová slova (red), Hex (#FF0000), RGB/RGBA (rgba(255, 0, 0, 0.5)). *

Jiné: Klíčová slova (auto , none), URL (url('obrazek.png')).

B. Dědění (Inheritance) a Kaskádování (Cascading)

Řeší, jak se styly šíří a jak se řeší konflikty.

1. Dědění (Inheritance)

Některé vlastnosti rodiče se automaticky přenášejí na potomky. * **Dědí se:** font-family , color , line-height , text-align , font-size . * **Nedědí se:** margin , padding , border , width , height , background-color .

2. Kaskádování (Cascading)

Proces řešení konfliktů, když se na prvek vztahuje více pravidel pro stejnou vlastnost. Rozhoduje se dle priority:

Původ a důležitost (Origin & Importance):

!important přebije téměř vše (nedoporučuje se).

Autorské styly > Výchozí styly prohlížeče.

Pořadí priority (od nejnižší): Uživatelské styly prohlížeče < Uživatelské styly < Autorské styly < Autorské styly s !important < Uživatelské styly s !important .

Specifita (Specificity):

Čím přesnější selektor, tím vyšší váha.

Váha se počítá:

Inline styl: (1, 0, 0, 0)

ID selektor (#id): (0, 1, 0, 0)

Třída, pseudotřída, atribut (.class , :hover , [type="text"]): (0, 0, 1, 0)

Element/Tag, pseudoelement (div , ::before): (0, 0, 0, 1)

Pořadí v kódu (Order of Appearance):

Při stejné důležitosti a specifitě vyhrává pravidlo napsané později v kódu.

C. Blokový model CSS (Box Model)

Každý HTML element je obdélníkový box složený ze čtyř vrstev (od středu ven):

Obsah (Content): Vnitřek elementu (text, obrázky). Rozměry určují width a height .

Vnitřní okraj (Padding): Průhledný prostor mezi obsahem a rámečkem.

Rámeček (Border): Čára kolem paddingu.

Vnější okraj (Margin): Průhledný prostor vně rámečku, vytváří mezery mezi boxy.

Vlastnost `box-sizing`

Určuje, jak se počítají `width` a `height`. * `box-sizing: content-box;` **(výchozí):** Celková šířka = `width` (obsah) + `padding` + `border`.

Celková šířka = `width` + `padding-left` + `padding-right` + `border-left-width` + `border-right-width`

* `box-sizing: border-box;` **(moderní):** `width` je celková šířka elementu (obsah + padding + border).

Padding a border se "vtlačí" dovnitř. Doporučuje se globálně: * { `box-sizing: border-box;` } .

D. Typy zobrazení (Display Property)

Vlastnost `display` určuje chování elementu v toku dokumentu.

`display: block;`

Zabírá celou šířku, začíná na novém řádku.

Respektuje `width`, `height`, `margin`, `padding`, `border`.

Příklady: `div`, `p`, `h1`.

`display: inline;`

Zabírá jen šířku obsahu, nezpůsobuje zalomení řádku (vedle sebe).

Ignoruje `width`, `height`, vertikální `margin` a `padding`.

Příklady: `span`, `a`, `strong`.

`display: inline-block;`

Zobrazuje se vedle sebe (jako `inline`).

Respektuje `width`, `height`, `margin`, `padding`, `border` (jako `block`).

`display: flex;`

Nastaví element jako flex kontejner. Umožňuje flexibilní uspořádání potomků (flex položek) v jednom rozměru (řádek/sloupec).

Výkonný nástroj pro responzivní layouty.

E. Praktické použití

Responzivní layout, typografie, vizuální identita, přístupnost webu, oddělení vzhledu od struktury.

F. Nejčastější chyby u zkoušky

Neznalost rozdílu mezi `margin` a `padding`.

Přepisování stylů bez pochopení kaskády/specificity.

Zapomenutí na `box-sizing`.

Záměna chování `display: block;` a `display: inline;`.

Považování CSS za programovací jazyk.

G. Typické zkouškové otázky

Vysvětlete CSS kaskádu a specifitu.

Kaskáda: Mechanismus, který určuje, které CSS pravidlo se aplikuje při konfliktu. Rozhoduje se dle: Původu (`!important`), Specificity selektoru, Pořadí v kódu.

Specificita: Váha selektoru. Čím přesnější selektor, tím vyšší priorita. Počítá se: Inline (1,0,0,0) > ID (0,1,0,0) > Třída (0,0,1,0) > Element (0,0,0,1).

Popište box model.

Každý HTML element je obdélníkový box se 4 vrstvami (od středu ven): Content (obsah), Padding (vnitřní okraj), Border (rámeček), Margin (vnější okraj).

`box-sizing` : Ovlivňuje výpočet `width` / `height` . `content-box` (výchozí) znamená, že `width` je jen šířka obsahu. `border-box` znamená, že `width` zahrnuje obsah, padding i border.

Jak funguje `display: block` , `inline` a `flex` ?

`display: block;` : Zabírá celou šířku, začíná na novém řádku, respektuje `width` , `height` , `margin` , `padding` .

`display: inline;` : Zabírá jen šířku obsahu, zobrazuje se vedle sebe, ignoruje `width` , `height` a vertikální `margin` / `padding` .

`display: flex;` : Nastaví element jako flex kontejner, umožňující flexibilní uspořádání jeho přímých potomků (flex položek) v jednom rozměru (řádek/sloupec).

25. Webové aplikace: Klient, Server, JavaScript a DOM

Klíčové pojmy

webová aplikace, frontend, backend

JavaScript, DOM, event listener

AJAX, JSON, SPA, HTTP API

dynamické typování, prototypová dědičnost, jednovláknové, asynchronní, Event Loop

selektor (CSS)

A. Programování na straně serveru (Backend) vs. klienta (Frontend)

Moderní webové aplikace jsou distribuované systémy Klient-Server.

Programování na straně klienta (Frontend)

Kde: Běží v prohlížeči uživatele.

Tech: HTML (struktura), CSS (vzhled), JavaScript (chování).

Charakteristika: Stará se o UI, validaci vstupů, animace, dynamické překreslování bez načítání stránky.

Omezení: Kód je viditelný, nemá přímý přístup k souborům počítače ani k databázi.

Programování na straně serveru (Backend)

Kde: Běží na vzdáleném webovém serveru.

Tech: PHP, Python, Node.js, Java, C#, databáze (SQL, NoSQL).

Charakteristika: Zpracovává HTTP požadavky, provádí business logiku (ověření, DB operace), generuje HTML/JSON.

Výhody: Kód je skrytý, bezpečný, má plný přístup k databázi a souborům na serveru.

Komunikace mezi klientem a serverem

HTTP API: Klient posílá HTTP požadavky (GET, POST) na server.

JSON (JavaScript Object Notation): Běžný formát pro výměnu dat.

AJAX (Asynchronous JavaScript and XML): Umožňuje asynchronní komunikaci se serverem na pozadí bez nutnosti znovu načítat celou stránku, zlepšuje UX. Dnes primárně s JSON.

Single Page Application (SPA)

Webová aplikace, která načte jednu HTML stránku a dynamicky mění obsah pomocí JavaScriptu a AJAXu (API). Poskytuje plynulejší uživatelský zážitek.

B. Základy jazyka JavaScript (JS)

Multiparadigmatický, interpretovaný, dynamicky typovaný programovací jazyk, nativně podporovaný prohlížeči.

Dynamické typování: Typ proměnné (deklarované pomocí `let` nebo `const`) se určuje automaticky za běhu podle přiřazené hodnoty. `javascript let vek = 30; // number vek = "třicet"; // string`

Prototypová dědičnost: Objekty dědí vlastnosti přímo od jiných objektů (prototypů), ne od tříd (klíčové slovo `class` je syntaktický cukr).

Jednovláknové a asynchronní:

JS běží v prohlížeči v jediném hlavním vlákne (Main Thread).

Asynchronní programování zabráňuje zablokování UI při dlouhých operacích (např. AJAX):

Callback funkce: Spustí se po dokončení asynchronní operace.

Promises: Objekty reprezentující budoucí výsledek (pending, fulfilled, rejected).

`async/await` : Syntaktický cukr nad Promises pro čitelnější asynchronní kód.

Vše řídí **Event Loop**, který neustále kontroluje frontu událostí a callbacků a spouští je, když je hlavní vlákno volné.

C. DOM (Document Object Model) a přístup k prvkům

DOM: Stromová struktura objektů v paměti prohlížeče, vytvořená z HTML/XML dokumentu. Poskytuje programové rozhraní (API) pro přístup a manipulaci se strukturou, obsahem a styly dokumentu pomocí JavaScriptu.

Přístup k prvkům stránky (Selekce uzlů):

```
document.getElementById('id_prvku')  
document.getElementsByClassName('navez_tridy') (HTMLCollection)  
document.querySelector('.css_selektor') (první odpovídající prvek)  
document.querySelectorAll('div.polozka') (NodeList všech odpovídajících prvků)
```

Manipulace s elementy: `javascript const nadpis = document.querySelector('#hlavni-nadpis');`
`nadpis.textContent = 'Nový text'; // Změna textu nadpis.innerHTML = '<em>Nový</em> text'; // Změna HTML`
`nadpis.style.color = 'blue'; // Změna inline CSS nadpis.classList.add('aktivni-styl');` // Správa CSS
`tříd nadpis.setAttribute('data-info', 'value');` // Přidání atributu

Reakce na události (Event Handling):

JS reaguje na akce uživatele (klik, stisk klávesy) pomocí posluchačů událostí. `javascript const tlacitko =`
`document.querySelector('#odesilaci-tlacitko');` `tlacitko.addEventListener('click', function(event) {`
`alert('Kliknuto!'); });`

Typické zkouškové otázky

Co je DOM? Programové API pro HTML/XML dokumenty. Reprezentuje dokument jako strom uzlů (elementy, atributy, text), umožňuje JS manipulovat se strukturou, obsahem a styly.

Jak JavaScript reaguje na události? Pomocí "event handlingu". Programátor registruje `event listener` (např. `addEventListener()`) k DOM elementům. Při události (klik, stisk klávesy) prohlížeč spustí asociovanou JS funkci (callback).

Jak webová aplikace komunikuje se serverem? Přes HTTP protokol. Klient posílá HTTP požadavky (GET, POST) na server. Server zpracuje a odpoví HTTP odpovědí (HTML, CSS, JS, JSON). Moderní aplikace využívají AJAX pro asynchronní komunikaci s REST API, dynamicky aktualizují obsah bez načítání stránky.

Zde je zkrácená verze (tahák) k otázce 26:

26. Jazyk PHP. Problematika uchovávání stavových informací, cookies.

A. Jazyk PHP

Klíčové pojmy: PHP (Hypertext Preprocessor), server-side, dynamicky typovaný, interpretovaný, Zend Engine, PHP-FPM, HTML embedding.

Definition: Open-source skriptovací jazyk pro dynamické webové stránky. Dynamicky typovaný, interpretovaný (Zend Engine kompiluje do bytecode).

Princip fungování:

Klient požádá `.php` soubor. Webový server předá PHP interpretu (PHP-FPM).

PHP kód v `<?php ... ?>` se zpracuje.

Generuje dynamický obsah (DB interakce, formuláře) a vkládá do HTML (`echo / print`).

Server odešle finální HTML (bez PHP kódu) klientovi.

Využití: WordPress, Laravel, Symfony.

B. Uchovávání stavových informací

Klíčové pojmy: Bezstavový protokol (HTTP), stavová informace, session management.

Problém: HTTP je bezstavový – server si nepamatuje předchozí interakce s klientem.

Nutnost stavu: Autentizace, nákupní košík, personalizace obsahu.

Řešení: Cookies a Relace (Sessions).

C. Cookies

Klíčové pojmy: `Set-Cookie` , `Cookie` , klient-side, `Max-Age` , `Expires` , `Path` , `Domain` , `HttpOnly` , `Secure` , `SameSite` , XSS, Session Hijacking, velikostní limit.

Definition: Malý textový záznam (klíč=hodnota) uložený klientem (webovým prohlížečem).

Fungování:

Zápis (server -> klient): Server pošle `Set-Cookie` hlavičku v HTTP odpovědi. Prohlížeč uloží.

Odesílání (klient -> server): Prohlížeč automaticky přibalí uložené cookie do `Cookie` hlavičky HTTP požadavku.

Vlastnosti a omezení:

Klient-side: Uživatel může mazat, zakázat, manipulovat.

Bezpečnostní rizika: Neukládat citlivé údaje. Náchylné k XSS (útok na cookie), Session Hijacking.

Velikost: Max 4 KB na cookie.

Metadata (příznaky):

`Max-Age` / `Expires` : Doba platnosti.

`HttpOnly` : Zabraňuje JavaScriptu v přístupu (ochrana před XSS).

`Secure` : Jen přes HTTPS spojení.

`SameSite` : Ochrana před CSRF (např. `Lax` , `Strict`).

D. Relace (Sessions) v PHP

Klíčové pojmy: Server-side úložiště, Session ID, `session_start()` , `PHPSESSID` , `$_SESSION` , session fixation.

Definition: Uchovávání stavových dat na straně serveru. Cookie slouží pouze k uložení **identifikátoru relace (Session ID)**.

Fungování:

Inicializace: `session_start()` : PHP vygeneruje unikátní Session ID (pokud neexistuje).

Server-side úložiště: Server uloží data relace (např. do souboru) pod Session ID. Přístupná přes superglobální pole `$_SESSION` .

Propojení s klientem: Server pošle Session ID klientovi jako speciální cookie (`PHPSESSID`).

Obnovení: Klient pošle `PHPSESSID` zpět. PHP najde data na serveru a obnoví `$_SESSION` .

Výhody:

Bezpečnost: Surová data jsou na serveru, klient má jen ID.

Kapacita: Bez limitu 4 KB jako u cookies.

Životní cyklus: Spravován serverem (garbage collection).

Zabezpečení:

`session_regenerate_id()` : Pravidelná změna Session ID (ochrana před Session Fixation).

Použití `HttpOnly` a `Secure` pro `PHPSESSID` cookie.

Dostatečná entropie Session ID.

Typické zkouškové otázky

Jak PHP zpracuje formulář?

Po odeslání formuláře (GET/POST) server přijme HTTP požadavek.

PHP skript (definovaný v `action` atributu) se spustí.

Data formuláře jsou automaticky naplněna do superglobálních polí `$_GET` nebo `$_POST` .

Skript data validuje, zpracuje (např. uloží do DB) a vygeneruje dynamickou HTML odpověď pro klienta.

Jaký je rozdíl mezi cookie a session?

Cookie: Malý datový záznam uložený na straně **klienta** (v prohlížeči). Server ji nastaví (`Set-Cookie`), prohlížeč ji automaticky posílá zpět (`Cookie`). Omezená velikost (4 KB), náchylné k manipulaci uživatelem nebo útokům (XSS).

Session (relace): Mechanismus pro uchování stavových dat na straně **serveru**. Klient u sebe v cookie (`PHPSESSID`) uchovává pouze unikátní **Session ID**. Server podle ID najde příslušná data relace (uložená na serveru) a zpřístupní je PHP skriptu (v `$_SESSION`). Sessions jsou bezpečnější pro citlivá data a nemají tak přísné velikostní omezení. Cookie tedy **identifikuje** relaci, zatímco relace **uchovává** skutečná data.

Jak zabezpečit autentizační cookie?

`HttpOnly` příznak: Zabraňuje JavaScriptu v přístupu k cookie (ochrana před XSS).

`Secure` příznak: Zajišťuje, že cookie bude odeslána pouze přes šifrované HTTPS spojení.

`SameSite` příznak (`Lax` nebo `Strict`): Chrání před CSRF útoky.

Krátká doba platnosti (`Max-Age`): Snižuje okno pro potenciální zneužití.

`session_regenerate_id()` : Pravidelná změna Session ID (ochrana před Session Fixation).

Ukládat pouze Session ID, ne citlivé informace přímo do cookie.

Volitelně: Validace IP adresy/User-Agenta na serveru.